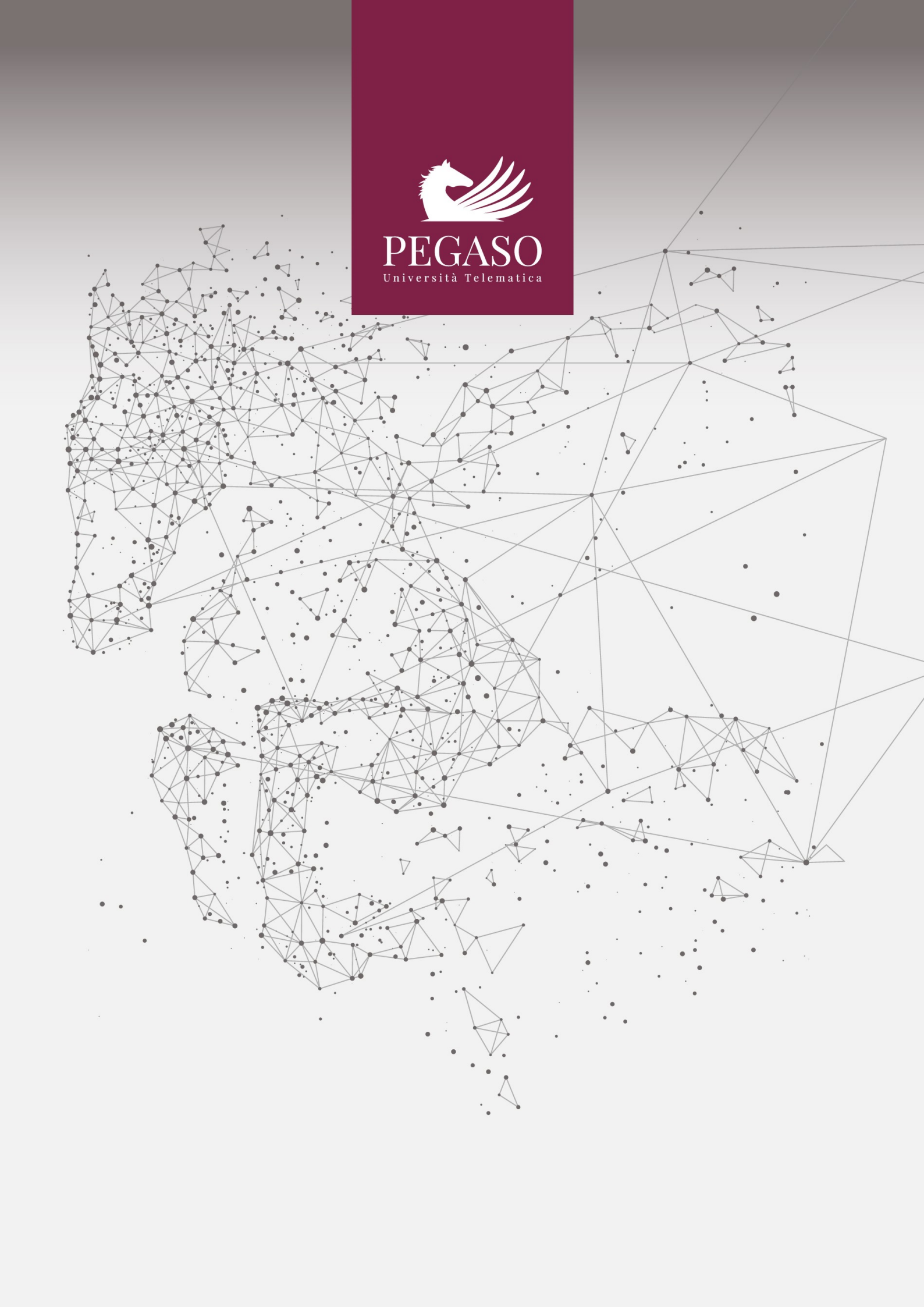

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. EXPRESS 3
2. MVC 4
3. STRUTTURA GENERALE 6
4. ROUTES 9
BIBLIOGRAFIA 12
SITOGRAFIA 13

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Express

Express.js (o semplicemente "Express") è un framework per Node.js che semplifica la creazione di applicazioni web. Express fornisce una serie di funzionalità per gestire le richieste HTTP, definire rotte, implementare middleware e utilizzare template engine per generare HTML dinamicamente.

Express è uno dei framework più utilizzati in Node.js per la creazione di applicazioni web, grazie alla sua semplicità, flessibilità e scalabilità. Express è stato progettato per essere minimalista e non impone una struttura di progetto specifica, consentendo agli sviluppatori di organizzare il codice in base alle proprie esigenze.

Le funzionalità principali di Express includono:

- **Gestione delle richieste HTTP**: Express semplifica la gestione delle richieste HTTP fornendo una serie di metodi per rispondere alle richieste GET, POST, PUT, DELETE e altre.
- **Definizione delle rotte**: Express consente di definire le rotte dell'applicazione utilizzando la sintassi app.METHOD(PATH, HANDLER), dove METHOD è il metodo HTTP, PATH è il percorso della rotta e HANDLER è una funzione che gestisce la richiesta.
- **Implementazione di middleware**: Express consente di implementare middleware, ovvero funzioni che vengono eseguite prima o dopo la gestione delle richieste HTTP. I middleware possono essere utilizzati per eseguire controlli di sicurezza, autenticazione, log, e altro ancora.
- **Utilizzo di template engine**: Express supporta diversi template engine, come Handlebars, EJS e Pug, per generare HTML dinamicamente.

In sintesi, Express.js semplifica la creazione di applicazioni web in Node.js fornendo un'API semplice e flessibile per gestire le richieste HTTP, definire rotte, implementare middleware e utilizzare template engine per generare HTML dinamicamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. MVC

MVC (Model-View-Controller) è un pattern architetturale utilizzato per separare le responsabilità del codice in tre parti distinte: il **Model**, la **View** e il **Controller**.

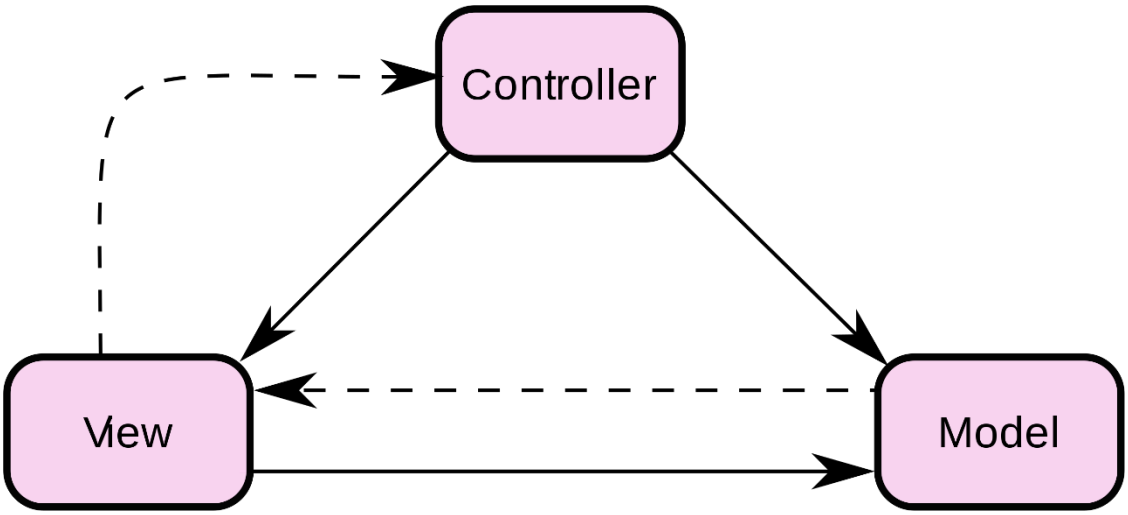


***Figura 1** – MVC*

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. Il Model non conosce nulla della View o del Controller, ma fornisce solo un'interfaccia per accedere ai dati.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. La View non ha alcuna conoscenza del Model o del Controller, ma fornisce solo un'interfaccia utente per interagire con l'applicazione.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. Il Controller riceve le richieste dell'utente dalla View, accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

L'utilizzo del pattern MVC ha diversi vantaggi, tra cui una maggiore separazione delle responsabilità del codice, una maggiore modularità, una maggiore flessibilità e una maggiore facilità di manutenzione e testabilità.

In Node.js, il pattern MVC può essere implementato utilizzando un framework web come Express.js. Ad esempio, si può utilizzare Express per gestire le richieste dell'utente (Controller), definire rotte e middleware per l'accesso ai dati dell'applicazione (Model) e utilizzare template engine come Handlebars o EJS per creare le View.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vediamo nel dettaglio i singoli componenti del pattern MVC e come possono essere implementati in Node.js.

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. In Node.js, il Model può essere implementato utilizzando una libreria di accesso ai database come Sequelize, Mongoose o Knex.js. Ad esempio, Sequelize è un ORM (Object-Relational Mapping) che permette di definire i modelli dei dati dell'applicazione e di interagire con il database utilizzando oggetti JavaScript.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. In Node.js, la View può essere implementata utilizzando un template engine come Handlebars, EJS o Pug. Ad esempio, Handlebars permette di creare template HTML dinamici utilizzando una sintassi simile a quella di Mustache.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. In Node.js, il Controller può essere implementato utilizzando le funzionalità di routing di Express.js. Ad esempio, si può utilizzare Express per definire le rotte dell'applicazione e le funzioni che gestiscono le richieste HTTP. In questo modo, il Controller accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Struttura generale

In generale, la struttura di un'applicazione Node.js che utilizza il pattern MVC prevede la seguente suddivisione del codice:

```
node-express-app/
  |- controllers/
  |    |- userController.js
  |    |- postController.js
  |- models/
  |    |- userModel.js
  |    |- postModel.js
  |- views/
  |    |- home.hbs
  |    |- user.hbs
  |    |- post.hbs
  |- public/
  |    |- css/
  |    |- js/
  |- routes/
  |    |- index.js
  |    |- userRoutes.js
  |    |- postRoutes.js
  |- app.js
  |- package.json
  |- README.md
```

In questa struttura, il file principale dell'applicazione è app.js, che avvia l'istanza di Express e definisce la configurazione principale dell'applicazione.

- La cartella **controllers**/ contiene i file dei controller, che definiscono le funzioni che gestiscono le richieste HTTP e accedono ai dati del modello per elaborare le risposte.
- La cartella **models**/ contiene i file dei modelli, che definiscono i dati dell'applicazione e le operazioni per accedere e manipolare questi dati.
- La cartella **views**/ contiene i file dei template engine, utilizzati per generare le viste dell'applicazione.
- La cartella **public**/ contiene i file statici come fogli di stile CSS e script JavaScript.
- La cartella **routes**/ contiene i file delle rotte dell'applicazione, che definiscono le rotte utilizzate dall'applicazione e collegano i controller e i modelli necessari.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Il file **index.js** nella cartella routes/ è utilizzato per definire le rotte principali dell'applicazione e collegare le rotte definite nei file userRoutes.js e postRoutes.js.
- Il file **package.json** definisce le dipendenze e le configurazioni dell'applicazione.
- Il file README.md contiene la documentazione dell'applicazione.

Naturalmente, questa è solo una struttura di base e può essere personalizzata in base alle esigenze specifiche dell'applicazione che si sta sviluppando.

- una cartella "**models**" per la definizione dei modelli dei dati
- una cartella "**views**" per i template HTML
- una cartella "**controllers**" per la definizione delle funzioni di routing di Express.js.

Ecco la struttura di un file app.js:

```
// Importa le dipendenze necessarie
const express = require('express');
const bodyParser = require('body-parser');
const path = require('path');

// Crea un'istanza dell'applicazione Express
const app = express();

// Configura il middleware per il parsing del corpo delle richieste HTTP
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

// Configura la cartella "public" per i file statici
app.use(express.static(path.join(__dirname, 'public')));

// Definisci le rotte dell'applicazione
const indexRouter = require('./routes/index');
const userRouter = require('./routes/user');
const postRouter = require('./routes/post');
app.use('/', indexRouter);
app.use('/users', userRouter);
app.use('/posts', postRouter);

// Gestisci gli errori
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Internal server error');
});

// Avvia il server sulla porta specificata
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  console.log(`Server avviato sulla porta ${PORT}`);
});
```

Di seguito una descrizione schematica del codice presentato:

- const express = require('express'): importa il modulo Express per creare un'istanza dell'applicazione.
- const bodyParser = require('body-parser'): importa il modulo body-parser per il parsing del corpo delle richieste HTTP.
- const path = require('path'): importa il modulo path per la gestione dei percorsi dei file.
- const app = express(): crea un'istanza dell'applicazione Express.
- app.use(bodyParser.urlencoded({ extended: true })): configura il middleware per il parsing del corpo delle richieste HTTP in formato URL-encoded.
- app.use(bodyParser.json()): configura il middleware per il parsing del corpo delle richieste HTTP in formato JSON.
- app.use(express.static(path.join(__dirname, 'public'))): configura la cartella public come cartella dei file statici. Questo middleware viene utilizzato per fornire i file statici come fogli di stile CSS, immagini e script JavaScript alle pagine HTML.
- const indexRouter = require('./routes/index'): importa il router per la rotta principale.
- const userRouter = require('./routes/user'): importa il router per le rotte degli utenti.
- const postRouter = require('./routes/post'): importa il router per le rotte dei post.
- app.use('/', indexRouter): utilizza il router per la rotta principale.
- app.use('/users', userRouter): utilizza il router per le rotte degli utenti.
- app.use('/posts', postRouter): utilizza il router per le rotte dei post.
- app.use((err, req, res, next) => {...}): middleware per la gestione degli errori. In caso di errore, viene stampato un messaggio di errore sulla console e viene restituita una risposta HTTP con lo stato 500 e il messaggio "Internal server error".
- const PORT = process.env.PORT || 3000: definisce la porta sulla quale il server HTTP ascolta le richieste in arrivo. Se la variabile d'ambiente PORT non è definita, il server ascolta sulla porta 3000.
- app.listen(PORT, () => {...}): avvia

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Routes

Le rotte (o routes) rappresentano uno dei concetti fondamentali di Express e permettono di definire come l'applicazione deve gestire le richieste HTTP provenienti dal client.

Le rotte definiscono un percorso, il metodo HTTP corrispondente e il callback da eseguire per gestire la richiesta. Il percorso può essere una stringa statica, un'espressione regolare o una combinazione di entrambi.

Si possono definire rotte per i diversi metodi HTTP, come ad esempio GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD, etc. Per definire una rotta per un metodo diverso da GET, si può utilizzare la corrispondente funzione app.post(), app.put(), app.delete(), etc.

Le rotte possono essere definite in file separati per una migliore organizzazione del codice: si possono definire i router che gestiscono le rotte per un gruppo di funzionalità correlate, e poi collegare questi router all'applicazione principale utilizzando il metodo app.use().

Se volessimo inserire la gestione delle rotte direttamente nel file principale app.js dovremmo commentare la sezione relativa alle rotte "esterne" ed inserire ad es. il seguente codice:

```
app.get('/', (req, res) => {
  res.send('Hello World!')
});
```

In questo esempio, la funzione app.get() definisce la rotta per le richieste HTTP GET all'URL /users. La funzione res.send() invia una risposta HTTP con il messaggio "Hello World!".

Nella programmazione web, una callback rappresenta una funzione che viene eseguita in risposta a un evento: nel caso di Express, una callback viene eseguita quando viene ricevuta una richiesta HTTP, e riceve come parametri l'oggetto richiesta req e l'oggetto risposta res.

L'oggetto richiesta req contiene tutte le informazioni sulla richiesta HTTP in arrivo, come ad esempio l'URL richiesto, i parametri passati nell'URL, il metodo HTTP utilizzato, i dati del corpo della richiesta, gli header della richiesta, etc; l'oggetto risposta res rappresenta la risposta HTTP che verrà inviata al client. Tramite l'oggetto risposta res, si possono impostare lo status code della risposta, gli header della risposta e il corpo della risposta.

Quando una richiesta HTTP viene gestita da una callback, la callback utilizza l'oggetto risposta res per inviare una risposta HTTP al client: ad esempio, si potrebbe utilizzare il metodo res.send() per inviare una risposta HTTP con un messaggio di testo, oppure il metodo res.sendFile() per inviare un file.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La callback può anche utilizzare l'oggetto richiesta req per accedere ai dati della richiesta. Ad esempio, si può utilizzare req.params per accedere ai parametri dell'URL, req.body per accedere ai dati del corpo della richiesta, o req.query per accedere ai parametri della richiesta in formato URL-encoded.

La rotta appena analizzata può anche essere definita in file separato per una migliore organizzazione del codice: si possono definire i router che gestiscono le rotte per un gruppo di funzionalità correlate, e poi collegare questi router all'applicazione principale utilizzando il metodo app.use().

Pertanto, è possibile re-inserire nel file app.js il seguente codice:

```
const indexRouter = require('./routes/index');
app.use('/', indexRouter);
```

Sarà poi necessario creare un file index.js all'interno della cartella routes/ ed inserire (ad es.) il seguente codice:

```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('Hello World');
});

router.get('/:id', (req, res) => {
  const myId = req.params.id;
  res.send(`Hello World ${myId}`);
});

router.post('/', (req, res) => {
  const _body = req.body;
  console.log(_body);
  res.send('POST');
});

module.exports = router;
```

In questo file, si definiscono tre rotte per il router router:

- una per la richiesta HTTP GET all'URL /
- una per la richiesta HTTP GET all'URL /:id
- una per la richiesta HTTP POST all'URL /.

Le rotte definite utilizzano il metodo res.send() per inviare una risposta HTTP al client. Il metodo res.send() invia un messaggio di testo come risposta HTTP al client.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il parametro dinamico :id viene utilizzato per catturare un valore dinamico dall'URL. I valori dei parametri dinamici possono essere recuperati utilizzando la proprietà req.params dell'oggetto richiesta req.

La rotta per la richiesta HTTP POST utilizza req.body per recuperare il corpo della richiesta HTTP. Il middleware body-parser deve essere configurato per l'applicazione per poter utilizzare questa funzionalità. In questo caso, il corpo della richiesta viene stampato sulla console utilizzando console.log().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).